

array_push() — add elements to the end

Adds one or more values to the **end** of an existing array.

Example:

```
$a = [1, 2];  
array_push($a, 3, 4);
```

Result: [1, 2, 3, 4]

Explanation:

- PHP takes the array \$a.
- Appends 3 then 4 after the last element.
- The original array is modified.

Notes / Tips:

- Same behavior as JavaScript push().
- Returns the new array length, not the array itself.

~~~~~

## array\_pop() — remove the last element

Removes the **last item** from an array and returns it.

Example:

```
$a = [1, 2, 3];  
$removed = array_pop($a);
```

Result: removed = 3, array = [1, 2]

Explanation:

- PHP removes the element at the highest index.
- That value is returned and can be stored.
- The array becomes shorter by one element.

Notes / Tips:

- Same idea as JavaScript pop( ).

---

## array\_unshift() — add elements to the beginning

Adds one or more values to the **start** of an array.

Example:

```
$a = [2, 3];  
array_unshift($a, 1);
```

Result: [1, 2, 3]

Explanation:

- PHP inserts 1 at index 0.
- All existing elements are shifted forward.

Notes / Tips:

- Same as JavaScript unshift().

---

## array\_shift() — remove the first element

Removes the **first item** of an array and returns it.

Example:

```
$a = [1, 2, 3];  
$removed = array_shift($a);
```

Result: removed = 1, array = [2, 3]

Explanation:

- PHP removes index 0.
- Remaining elements shift left.

Notes / Tips:

- Same as JavaScript shift().
-

## array\_merge() — combine arrays

Joins multiple arrays into a single new array.

Example:

```
$a = [1, 2];  
$b = [3, 4];  
$result = array_merge($a, $b);
```

Result: [1, 2, 3, 4]

Explanation:

- PHP takes values from \$a, then \$b.
- Numeric keys are re-indexed.
- Original arrays are unchanged.

~~~~~

array_slice() — copy part of an array

Extracts a section of an array **without modifying** the original.

Example:

```
$a = [1, 2, 3, 4];  
$result = array_slice($a, 1, 2);
```

Result: [2, 3]

Explanation:

- Start at index 1 (value 2).
- Take 2 elements.
- \$a stays the same.

~~~~~

## array\_splice() — remove and/or insert

Removes elements and optionally inserts new ones **in place**.

Example:

```
$a = [1, 2, 3, 4];  
array_splice($a, 1, 2, [99]);
```

**Result: [1, 99, 4]**

Explanation:

- Start at index 1 (value 2).
- Remove 2 elements → [2, 3].
- Insert 99 at the same position.
- Original array is changed.

Notes / Tips:

- Same role as JavaScript splice().
- Only returns the removed elements

~~~~~

array_filter() — keep elements that match a condition

Filters array values using a callback function.

Example:

```
$a = [1, 2, 3, 4];  
$result = array_filter($a, fn($x) => $x > 2);
```

Result: [3, 4]

Explanation:

- fn() a way to declare an arrow function
- Each value is passed to the callback.
- If callback returns true → value is kept.
- If false → value is removed.

~~~~~

## array\_map() — transform each element

Applies a function to every element and returns a new array.

Example:

```
$a = [1, 2, 3];  
$result = array_map(fn($x) => $x * 2, $a);
```

Result: [2, 4, 6]

Explanation:

- Each element is multiplied by 2.
- A new array is returned.

~~~~~

array_reduce() — reduce array to one value

Combines all values into a single result.

Function parameters names (Important)

First parameter → Accumulator

- Also called: **carry, accumulated value**
- PHP's official term: **\$carry**

This is the value that **keeps the running result** between iterations.

Second parameter → Current value

- Also called: **current element**
- PHP's official term: **\$item**

This is the current element from the array during each iteration.

Third parameter → initial value = (0)

It should be looking like this:

```
array_reduce(array, (accumulator, current value) , initial value)
```

Example:

```
$a = [1, 2, 3];  
$result = array_reduce($a, fn($sum, $n) => $sum + $n, 0);
```

Result: 6

Explanation:

- \$sum starts at 0.
- Each value is added to \$sum.
- Final value is returned

~~~~~

## in\_array() — check if value exists

Checks whether a value is present in an array.

Example:

```
$a = [1, 2, 3];  
$result = in_array(2, $a);
```

**Result: true**

Explanation:

- Returns true if value is found.
- Returns false otherwise.
- User `var_dump($variable) // bool(true) or bool(false)`

|                                           |    |                                          |
|-------------------------------------------|----|------------------------------------------|
| <div><b>Result:</b><br/>bool(false)</div> | or | <div><b>Result:</b><br/>bool(true)</div> |
|-------------------------------------------|----|------------------------------------------|

~~~~~

array_search() — find key/index of a value

Finds the position of a value in an array.

Example:

```
$a = ['a', 'b', 'c'];  
$result = array_search('b', $a);
```

Result: 1

Explanation:

- Returns the key where the value exists.
- Returns false if not found.

~~~~~

## array\_keys() — get all keys

Returns all keys from an array.

Example:

```
$a = ['name' => 'Taha', 'age' => 20];  
$result = array_keys($a);
```

Result: ['name', 'age']

~~~~~

array_values() — get all values only

Removes keys and keeps values.

Example:

```
$a = ['name' => 'Taha', 'age' => 20];  
$result = array_values($a);
```

Result: ['Taha', 20]

~~~~~

## count() — count elements

Counts how many elements are in an array.

Example:

```
$a = [10, 20, 30];  
echo count($a);
```

Result: 3

~~~~~

sort() — sort values ascending

Sorts values from smallest to largest.

Example:

```
$a = [3, 1, 2];  
sort($a);
```

Result: [1, 2, 3]

~~~~~

## rsort() — sort values descending

Sorts values from largest to smallest.

Example:

```
$a = [3, 1, 2];  
rsort($a);
```

Result: [3, 2, 1]

~~~~~

asort() — sort values, keep keys

Sorts an associative array by values.

Example:

```
$a = ['a' => 3, 'b' => 1];  
asort($a);
```

Result: ['b' => 1, 'a' => 3]

~~~~~

## ksort() — sort by keys

Sorts an array based on its keys.

Example:



```
$a = ['b' => 2, 'a' => 1];  
ksort($a);
```

Result: ['a' => 1, 'b' => 2]

~~~~~

array_unique() — remove duplicate values

Removes repeated values from an array.

Example:

```
$a = [1, 2, 2, 3];  
$result = array_unique($a);
```

Result: [1, 2, 3]

~~~~~

## array\_reverse() — reverse array order

Reverses the order of elements.

Example:

```
$a = [1, 2, 3];  
$result = array_reverse($a);
```

Result: [3, 2, 1]

## **Golden Rule**

Functions like *push*, *pop*, *shift*, *unshift*, *splice*, *sort* **MODIFY** the array.

Functions like *slice*, *map*, *filter*, *reduce*, *reverse* **RETURN** a new array.